

R Language

Set the directory

```
getwd() #to know the working directory
```

```
setwd("G:\\Document\\PDF\\Department\\L1- S2\\Theory\\COMPS(Computer Science)\\R\\My creation")
```

```
list.files() #show the files in the directory
```

Conditional sentence

```
> a<- 10
```

```
> b<- 15
```

```
> if(a>b){print("a is greater than b")} else if(a==b){print("a and b are equal")} else{print("b is greater than a")}
```

Loop

```
> for(x in 1:10)
```

```
+ {print(x)}
```

Function

```
> avarage <- function(x,y,z){
```

```
+     ava = (x+y+z)/2
```

```
+     print(ava)
```

```
+ }
```

Data type

```
a <- 10
```

```
typeof(a)
```

```
b <- "swarup"
```

```
typeof(b)
```

R Language

```
c <- TRUE
```

```
typeof(c)
```

```
d <- 10L
```

```
typeof(d)
```

```
e <- 5i
```

```
typeof(e)
```

Data length

```
Marks <- c(89,87,89)
```

```
typeof(Marks)
```

```
length(Marks)
```

```
Marks[c(1,3)]
```

Sub setting

```
Number <- 1:100
```

```
Number
```

```
odd_number <- seq(1,99,by=2)
```

```
odd_number
```

```
from100to_1 <- seq(100,1,-1)
```

```
from100to_1
```

```
odd_number[10:20]
```

```
odd_number[c(12,32,50,99)]
```

R Language

```
odd_number[odd_number>20]
```

```
odd_number>20
```

Example 1: Calculating Circumference

```
ra <- 10:20
```

```
circumference <- 2*pi*ra
```

```
circumference
```

```
typeof(circumference)
```

```
circumference1 <- as.integer(circumference)
```

```
circumference1
```

```
typeof(circumference1)
```

```
circumference2 <- as.character(circumference)
```

```
circumference2
```

```
typeof(circumference2)
```

```
circumference3 <- as.logical(circumference)
```

```
circumference3
```

```
typeof(circumference3)
```

Example 2: BMI calculation

```
family_name <- c("Nurul","Mokhles","Sani")
```

```
Height <- c(155,150,120) #in cm
```

```
Weight <- c(55,65,45) #in kg
```

R Language

```
Family_info <- data.frame(family_name,Height,Weight)
```

```
plot(Family_info)
```

```
plot(Height~Weight,Family_info)
```

```
#creating a new column of BMI
```

```
Family_info$BMI <- Family_info$Weight/(Family_info$Height*0.01)^2
```

```
Family_info
```

```
#creating a category
```

```
Family_info$Health_level <- ifelse(Family_info$BMI<=18.5,"UNDERWEIGHT",ifelse(Family_info$BMI<=24.9,"NORMAL  
WEIGHT",ifelse(Family_info$BMI<=29.9,"OVERWEIGHT","OBESITY")))
```

```
Family_info
```

Creating a dataset in r

```
> student_data <- data.frame (name=c("shadhin","joy","durjoy"),roll=c(23,45,32))
```

```
creating dataset with metrix
```

```
met <- matrix(c(23,43,24,24,23,34,32,35,32,35,35,35,34,32,36), nrow=5,ncol=3,byrow=TRUE)
```

```
met
```

```
is.data.frame(met)
```

```
length(met)
```

```
creating with rep function
```

```
data_rep <- data.frame(method=rep(c("No 1","No 2","No 3"),each=4),
```

```
marks=c(1,2,3,4,5,6,7,8,9,1,2,3)
```

```
)
```

R Language

Data filter

```
data[data$roll==32 ,]
```

Adding a new row/grade

```
>data$grade <- ifelse(data$Mark>=80 , "A+" ,ifelse(data$Mark>=75 , "A" ,ifelse(data$Mark>=70 , "A-" ,ifelse(data$Mark>=65 , "B+" ,ifelse(data$Mark>=60 , "B" ,ifelse(data$Mark>=55 , "B-" ,ifelse(data$Mark>=50 , "C" , "F" ) ) ) ) ) ) )
```

Creating a CVS file through excel file...

Read file and basic things

```
Farmer_data<-read.csv("name of the file")
```

```
Farmer_data #to show the dataset
```

```
print(is.data.frame(data))
```

```
print(nrow(data)) #no of row in the dataset
```

```
print(ncol(data)) #no of column in the dataset
```

```
Farmer_data #show dataset
```

```
head(Farmer_data) #show first 6 row
```

```
tail(Farmer_data) #show last 6 row
```

```
summary(Farmer_data) #summary of the dataset
```

```
names(Farmer_data) #give the column of dataset
```

```
Farmer_data$Farm_Size #showing the column value of a dataset
```

```
length(Farmer_data) #showing the length of farmer data , how many collaum are there
```

```
length(Farmer_data $Region) #the length of region
```

Mean, Median, Mode

R Language

#mean Calculation

The average of the dataset

```
mean(Farmer_data$Revenue)
```

```
mean(Farmer_data $Rainfall,na.rm = TRUE) #here na.rm gives the command that calculate the mean without the missing value
```

#Median Calculation

The median is the middle of the set of numbers

```
median(Farmer_data$Revenue)
```

```
median(Farmer_data$Rainfall, na.rm = TRUE)
```

#Mode

Mode is the most frequently occurring value in the dataset

```
getmode <- function(v){
```

```
  return(names(which.max(table(v))))
```

```
}
```

```
mode_of_revenue <- getmode(Farmer_data$Revenue)
```

```
mode_of_revenue
```

Variance, Standard deviation

#variance calculation

```
var(Farmer_data$Revenue)
```

#Standard deviation calculation

```
sd(Farmer_data$Revenue)
```

R Language

Quartile

(providing insights into the distribution of the data)

```
dataset_q <- c(1,2,3,4,5,6,7,8,9,10,11,12,13,14,15)
```

```
quantile(dataset_q, probs = c(0.25, 0.5, 0.75))
```

Missing value

#find missing value

```
is.na(Farmer_data$Yield)
```

#count missing value

```
sum(is.na(Farmer_data$Yield))
```

#calculation of mean if the column has missing value

```
mean(Farmer_data$Yield, na.rm = TRUE)
```

#Omit missing Value

```
na.omit(Farmer_data)
```

#Fill the missing data with mean

```
mean_of_rainfall <- mean(Farmer_data$Rainfall, na.rm = TRUE)
```

```
Farmer_data[is.na(Farmer_data$Rainfall), "Rainfall"] <- mean_of_rainfall
```

```
print(Farmer_data)
```

Working with missing value with package

#visualizing the missing data using VIM package

R Language

```
library(VIM)
```

```
aggr(Farmer_data)
```

```
aggr(Farmer_data$Yield)
```

```
#fill missing data
```

```
library(mice)
```

```
imp <- mice(Farmer_data)
```

```
complete_farmer_data <- complete(imp)
```

```
#visualizing the dataset after filling the missing value
```

```
aggr(complete_farmer_data)
```

```
aggr(Farmer_data)
```

```
print(complete_farmer_data)
```

```
print(Farmer_data)
```

Line graph

```
> v <- c(13,12,45,34,56,67,89,78,56,78,90)
```

```
> plot(v,type = "p") --> this is just create point
```

```
> plot(v,type = "l") --> this is just create line
```

```
> plot(v,type = "o") --> this is create both line and point
```

```
WITH THE HEADING:
```

```
> days = c(10,20,30,40,50,60,70,80,90)
```

```
> income <- c(1000,3000,4500,6700,7800,8900,9900,11000,12000)
```

```
> plot(days,income,type = "o",main = "Salary",xlab = "days",ylab = "income", col="green")
```


R Language

MULTIPLE GRAPH

```
> f <- c(3,5,2,7,4,8,9,4)
```

```
> g <- c(4,6,8,6,7,4,7,5)
```

```
> j <- c(3,5,6,5,8,9,5,8)
```

```
> plot(f, type="o",main="multiple graph",xlab="days",ylab="time",col="blue")
```

```
> lines(g, type="o",col="green")
```

```
> lines(j, type="o",col="red")
```

Bar graph

CREATE THE MATRIX OF THE VALUES

```
>Values <- matrix(c(2, 9, 3, 11, 9, 4, 8, 7, 3, 12, 5, 2, 8, 10, 11), nrow = 3, ncol = 5, byrow = TRUE)
```

CREATE THE BAR CHART

```
>barplot(Values, main = "Total Revenue", names.arg = months, xlab = "Month", ylab = "Revenue", col = colors, beside = TRUE)legend
```

ADD THE LEGEND TO THE CHART

```
>legend("topleft", regions, cex = 0.7, fill = colors)
```

ggplot2

```
library(ggplot2)
```

```
data <- data.frame(
```

```
  x = c(1, 2, 3, 4, 5),
```

```
  y = c(3, 5, 2, 8, 7)
```

```
)
```

No 2:

```
ggplot(data,aes(x,y))+
```

R Language

```
geom_line(color = "blue")+
```

```
geom_point(size=3)+
```

```
labs(title = "Practice of ggplot",subtitle="add any subtitle hare",
```

```
caption= "add any caption if needed",x = "x axis", y= "y axis")+
```

```
theme_classic()
```

```
ggplot(data,aes(feed,weight, fill=feed, color=feed, shape = feed))+
```

```
geom_violin(alpha=0.3)+
```

```
geom_boxplot(alpha=0.5)+
```

```
geom_jitter(width=0.3)+
```

```
theme_classic()+
```

```
theme(legend.position="bottom")+
```

```
labs(x="feed(type)", y="Weight in grams",
```

```
title=" chickweight",
```

```
subtitle="feed and weiht response",
```

```
caption= " data taken form r base",
```

```
fill="", color="", shape="")
```

ttest

```
roll <- c(23,45,66,34,54)
```

```
markss <- c(87,89,80,98,87)
```

```
student_data <- data.frame(roll,markss)
```

```
student_data$group <- ifelse(student_data$roll < 50, "Group1", "Group2")
```

```
table(student_data$group)
```

```
ttest <- t.test(markss ~ group,data = student_data)
```

```
ttest
```

R Language

One way ANOVA

```
Stu_data <- data.frame(  
  Method = rep(c("A", "B", "C"), each = 5),  
  Score = c(85, 90, 88, 87, 84, # Method A  
            78, 76, 80, 75, 79, # Method B  
            92, 94, 95, 91, 93) # Method C  
)  
  
anova_result <- aov(Score ~ Method, data = Stu_data)  
  
summary(anova_result)
```

*Average Order Value (AOV)

- aov: A function in R for conducting ANOVA.
- Score: The dependent variable or the response variable.
- Method: The independent variable or the factor whose levels you are

Look at the p-value in the output:

- If p-value < 0.05, reject the null hypothesis (there is a significant difference between the groups).
- If p-value $\geq$ 0.05, fail to reject the null hypothesis (there is no significant difference).

The null hypothesis (H_0) is the default assumption that there is no difference between the groups.

AgroR

1. DIC() - Completely Randomized Design (CRD)

This function analyzes data from experiments with a completely randomized design.

```
with(eod,DIC(treatment,yield,addmean=TRUE,ylab = "yield per hector" ,fill = 'trat',quali = TRUE ))
```

R Language

- Same letter shows that there are no significance difference between treatment
- Here in ylab should give the response of the experiment.

2. DBC() - Randomized Complete Block Design (RCBD)

This function analyzes data from experiments arranged in randomized blocks.

```
with(eod,DBC(treatment,rep,yield,ylab = "yield per hactor",fill = 'trat',quali =TRUE))
```

here rep is a block

3. FAT2DIC() - Two-Factor Analysis in CRD

Analyzes data from two-factor factorial experiments in a completely randomized design.

```
FAT2DIC(f1, f2, resp, quali = c(TRUE, TRUE), ...)
```

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 1")
```

```
list.files()
```

```
data <- read.csv("fertilizer_yield_data.csv")
```

```
library(dplyr)
```

```
overall_mean <- mean(data$yield, na.rm=TRUE)
```

```
summary_table <- data %>%
```

R Language

```
filter(yield>2)%>%  
  
group_by(fertilizer) %>% # this is needed to calculate the average yield for each group  
  
summarise(avg_yeild = mean(yield, na.rm= TRUE)) %>% #the result is a new coloum  
  
mutate(yeild_level = ifelse(avg_yeild>overall_mean,"high","low")) %>%  
  
arrange(desc(avg_yeild))
```

#to save the file

```
write.csv(summary_table, "summary table.csv",row.names = FALSE)
```

#lesson for the summarise function. Can use more function in the summarise function.

```
new <- data%>%  
  
group_by(fertilizer) %>%  
  
summarise(  
  
  avg_yield = mean(yield, na.rm=TRUE),  
  
  max_yield = max(yield, na.rm = TRUE),  
  
  yield_sd = sd(yield, na.rm = TRUE),  
  
  unique_yield = n_distinct(yield, na.rm=TRUE)  
  
)
```

#use it to remove the unwanted column in the dataset

```
data %>%  
  
  select(fertilizer,yield)
```

#for rename the data column

```
data%>%  
  
  rename(fert= fertilizer)
```

#for just distinct value

```
data%>%  
  
  distinct()
```

#let's learn the case_when function. It will replace the ifelse , which can be very chaotic

R Language

```
data%>%  
  mutate(yield2= yield*5) %>%  
  mutate(yield_level=case_when(  
    yield2 < 10 ~ "Low",  
    yield2 < 15 ~ "moderate",  
    yield2 < 17 ~ "high",  
    TRUE ~ "durjoy"  
  ))
```

#the thing in the dplyr library is they do the first line then go to the second line. and then the third line..

DAY 1

```
data <- read.csv("fertilizer_yield_data.csv")
```

Data Wrangling with dplyr library

dplyr library is they do the first line then go to the second line. and then the third line.

```
library(dplyr)
```

```
overall_mean <- mean(data$yield, na.rm=TRUE)
```

```
summary_table <- data %>%
```

```
  filter(yield>2)%>%
```

```
  group_by(fertilizer) %>% #group the column on distinct data..
```

```
  summarise(avg_yield = mean(yield, na.rm= TRUE)) %>% #the result add a new column and they remove all the preexisting data.
```

```
  mutate(yield_level = ifelse(avg_yield>overall_mean,"high","low")) %>% #this is mostly use to add a new column from the dataset here.
```

R Language

```
arrange(desc(avg_yield))
```

Save the file as an csv format

```
write.csv(summary_table, "summary table.csv", row.names = FALSE)
```

Summarise function can add multiple condition and it will give the new column every time.

```
new <- data%>%
```

```
  group_by(fertilizer) %>%
```

```
  summarise(
```

```
    avg_yield = mean(yield, na.rm=TRUE),
```

```
    max_yield = max(yield, na.rm = TRUE),
```

```
    yield_sd = sd(yield, na.rm = TRUE),
```

```
    unique_yield = n_distinct(yield, na.rm=TRUE)
```

```
  )
```

Remove the unwanted column from the dataset

```
data %>%
```

```
  select(fertilizer, yield) #just show the written column.
```

Rename the data column

```
data%>%
```

```
  rename(fert= fertilizer)
```

Just distinct value

```
data%>%
```

```
  distinct()
```

Case_when

```
data%>%
```

R Language

```
mutate(yield2= yield*5) %>%
```

```
mutate(yield_level=case_when(
```

```
yield2 < 10 ~ "Low",
```

```
yield2 < 15 ~ "moderate",
```

```
yield2 < 17 ~ "high",
```

```
TRUE ~ "durjoy"
```

```
))
```

DAY 2

```
data <- read.csv("agriculture_sample_dataset.csv")
```

```
library(ggplot2)
```

Barplot

```
ggplot(data, aes(Fertilizer,Yield,fill=Fertilizer)) +
```

```
geom_bar(stat = "summary" )+
```

```
facet_wrap(~Region)+ #very very useful function. it separate the plots for each region.
```

```
theme_minimal()+
```

```
labs(title = "something",x="something 1",y="something 2",caption="something 3") +
```

```
scale_fill_brewer(palette = "set7") #professional and readable plots for reports.
```

Boxplot

```
ggplot(data, aes(Fertilizer,Yield))+
```

```
geom_boxplot(fill= "lightgreen")
```

Line and Point

For line and point. with this you can show the time series visualization. it is good for checking the growth habit for the plant.

Line plot use color not fill. Fill is used for the other plot

```
ggplot(data, aes(Month,Yield))+ #for better result the two variable should be numerical
```


R Language

```
geom_line(col="green")+
```

```
geom_point(col="red")
```

Time series visualization

```
okk <- read.csv("G:\\time_series_agriculture - Copy.csv")
```

```
okk$Date <- as.Date(okk$Date) #this is so much important. it convert the text into date.
```

```
ggplot(okk,aes(Date,Temperature, colour = Region)) +
```

```
geom_line() +
```

```
labs(title = "Daily Temperature Trend by Region", x = "Date", y = "Temperature (°C)")
```

position = "dodge" and the facet_wrap difference

```
ggplot(data, aes(Fertilizer,Yield,fill= Region)) +
```

```
geom_bar(stat = "summary",position = "dodge" )
```

```
ggplot(data, aes(Fertilizer,Yield,fill= Fertilizer)) +
```

```
geom_bar(stat = "summary")+
```

```
facet_wrap(~Region)
```

Grouping and Plotting

```
#fistly start grouping the data on just based of the fertilizer
```

```
library(dplyr)
```

```
summary_data <- data%>%
```

```
group_by(Fertilizer) %>%
```

```
summarise(
```

```
mean_yield = mean(Yield, na.rm= TRUE),
```

```
median_yield = median(Yield, na.rm= TRUE),
```

```
sd_yield = sd(Yield , na.rm = TRUE)
```

```
)
```

R Language

#bar plot of mean yeild by fertilizer

```
ggplot(summary_data, aes(Fertilizer, mean_yield, fill = Fertilizer))+
```

```
geom_bar(stat="summary")+
```

```
labs(title = "Mean Yield by Fertilizer", x = "Fertilizer", y = "Mean Yield (kg/ha)") +
```

```
theme_minimal()
```

Grouping based on the multiple column + Visualization

#Bar Plot of Mean Yield by Fertilizer and Region

```
sad_multi <- data %>%
```

```
group_by(Fertilizer,Region) %>%
```

```
summarise(
```

```
yield_mean = mean(Yield, na.rm=TRUE)
```

```
)
```

```
ggplot(sad_multi, aes(Fertilizer,yield_mean,fill=Region))+
```

```
geom_bar(stat="summary", position = "dodge") +
```

```
labs(title = "Mean Yield by Fertilizer and Region", x = "Fertilizer", y = "Mean Yield (kg/ha)") +
```

```
theme_minimal()
```

Combining multiple variable into a single plot

```
ggplot(data, aes(Month, Yield, color = Fertilizer)) +
```

```
geom_line() +
```

```
facet_wrap(~ Region) +
```

```
labs(title = "Yield Trend by Region and Fertilizer", x = "Month", y = "Yield (kg/ha)") +
```

```
theme_minimal()
```

R Language

DAY 3

```
data <- read.csv("fertilizer_yield for anova.csv")
```

Anova model creation

```
model <- aov(Yield~Fertilizer, data=data)
```

Assumption check

Shows one way anova is reliable or not.

1. normality test

```
shapiro.test(residuals(model))
```

Interpretation:

- p value is more than 0.05 the residuals are normally distributed, can perform the one way ANOVA.
- p value is less than 0.05 the residuals are not normally distributed, performing one way ANOVA isn't reliable.

Normal data:

Distribution means wheather the data are not well distributed or not.

Example: Fertilizer A yield are fluctuating so much like (2,5,20,50). → not well distributed

Normally distributed data is (15,13,14,16)

2. homogenecity test

```
library(car)
```

```
leveneTest(Yield~Fertilizer, data= data)
```

Interpretation:

- P-Value is more than 0.05, the variances across groups are equal (homogeneous), perform the one-way ANOVA reliably.

R Language

- P-Value is less than 0.05 the variances across groups are not equal (not homogeneous), performing one-way ANOVA isn't reliable.

Homogeneity:

For understanding homogeneity need to understand variance. Variance means the spread of data in each group.

Example:

Group A: (13, 14, 15, 16) → small spread → low variance

Group B: (10, 20, 30, 40) → large spread → high variance

All groups have similar spread, the data is homogeneous. Ad one group is very spread out and others are tight, it's not homogeneous.

3. Diagnostic plots

`plot(model)`

1.Residuals vs Fitted .. Look for the random spread(no clear pattern)

2.Normal Q-Q .. Look for the point near line.(normality)

3.Scale-Location

4.Residuals vs Leverage

ANOVA

`summary(model)`

Interpretation:

- P-Value < 0.05 → ✓ Statistically significant. There is a significant difference in yield among fertilizer types.
- P-Value > 0.05 → ✗ Not significant. No evidence of difference in yield between fertilizer groups.

Kruskal-Wallis test

When the data is not normally distributed and the variances are not equal.

R Language

```
krus <- read.csv("non_normal_yield_data.csv")
```

```
kruskal.test(Yield~Fertilizer, data=krus)
```

- P-Value < 0.05 → There is a significant difference in yield between at least one pair of fertilizers.
- P-Value > 0.05 , there isn't significant difference in yield between at least one pair of fertilizers.

DAY 4

```
data <- read.csv("agriculture_posthoc_data.csv")
```

```
library(agricolae)
```

```
model <- aov(Yield~Treatment, data = data)
```

```
shapiro.test(residuals(model))
```

```
library(car)
```

```
leveneTest(Yield~Treatment, data = data)
```

Anova is reliable

```
summary(model)
```

Interpretation:

- P-Value < 0.05 → go for post hoc test
- P-Value > 0.05 → hypothesis failed

P-Value is 0.000543.so we can go for post hoc test.

Post hoc test

1. LSD test (Least Significant Difference)

```
lsd_result <- LSD.test(model, "Treatment", p.adj = "none")
```

```
lsd_result
```

Interpretation:

- \$means -> show the mean yield per fertilizer
- \$comparison -> signif (stars indicate significance) ..*** = highly significant ($p < 0.001$) if ** = $p < 0.01$ if * = $p < 0.05$

R Language

- \$Group -> Same letter= no significant difference,
Different letters = significantly different

Show the specific result:

```
lsd_result$means
```

```
lsd_result$groups
```

```
lsd_result$comparison
```

LSD limitation:

LSD test is more liberal (less strict), so it's: Good for few treatments (2–4).

Not good for many groups → higher chance of false positives.

Rule of Thumb

- ✓ 2–3 groups → LSD is OK, especially for early or internal analysis
- ✓ 3+ groups or final result/report → Prefer Tukey's HSD
- If unsure, go with Tukey — it's safer and accepted

LSD with the emmeans library

```
library(emmeans)
```

```
emmeans(model, pairwise ~ Treatment, adjust="none")
```

```
adjust="none" → LSD Test
```

2. Turkey's HSD test

```
library(agricolae)
```

```
turkey_result <- HSD.test(model, "Treatment", group = TRUE)
```

```
turkey_result
```

same interpretation as LSD test

HSD with emmeans

```
emmeans(model, pairwise ~ Treatment, adjust= "tukey", group=TRUE)
```

R Language

3. Duncan's Test

Install the agricolae package → Call the library → Run ANOVA

```
duncan_test <- duncan.test(model, "Treatment", group= TRUE)
```

```
duncan_test
```

Difference between the test:

1. Sensitivity = How easily the test finds a difference

- High sensitivity = Finds small differences
- Low sensitivity = Only finds big differences

2. Strictness = How careful the test is to avoid wrong results

- Strict test = Needs more evidence to say “groups are different”
- Less strict = Can say “different” even with less evidence (risky)

✓ Final Summary Table

Test	Use When...	Sensitivity	Strictness	Error Control	Most Used?
Tukey	Safe, journal standard, >3 groups	✗ Lower	✓ High	✓ Strong	✓ Yes
LSD	2–3 groups, quick idea, early research	✓ High	✗ Low	✗ None	✗ Less
Duncan	Grouping letters, medium strictness	✓ High	⚠ Medium	⚠ Some	⚠ Some agri

4. Dunn's test

Used for not normally distributed data..

First check make a ANOVA model → check normality by `shapiro.test(residuals(model))` → check the spread of data in every treatment by the `leveneTest` with the `car` library. After find this data is not normal then run the `kruskal-wallis` test.

```
kruskal.test(Yield~Treatment, data = data)
```

#if the p-value is significant means less than 0.05 then do the dunn's test

R Language

Dunn's test

```
library(FSA)
```

```
dunnTest(Yield~Treatment, data=data, method="bonferroni")
```

```
#Look at p.adj (adjusted p-values)
```

```
#If p.adj < 0.05 → significant difference
```

Function	Purpose	Package Required
LSD.test()	LSD post hoc test	agricolae
HSD.test()	Tukey's HSD test	agricolae
duncan.test()	Duncan's multiple range test	agricolae
kruskal.test()	Kruskal-Wallis test (non-parametric)	Base R (no package)
dunnTest()	Dunn's test (after Kruskal-Wallis)	FSA
shapiro.test()	Normality check	Base R

DAY 1

```
data <- read.csv("agriculture_two_way_anova.csv")
```

Two-Way ANOVA

```
model <- aov(Yield~Fertilizer*Irrigation, data= data)
```

```
summary(model)
```

```
              Df Sum Sq Mean Sq F value    Pr(>F)
Fertilizer      2    199    99.5     99.5 3.38e-08 ***
Irrigation      1    288   288.0    288.0 9.40e-10 ***
Fertilizer:Irrigation  2    117    58.5     58.5 6.48e-07 ***
Residuals     12      12     1.0
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

a. Main Effects

- **factor(Fertilizer):** if $p < 0.05$ → Fertilizer has a **significant effect** on yield.
- **Factor(Irrigation):** if $p < 0.05$ → Irrigation also has a **significant effect** on Yield.

R Language

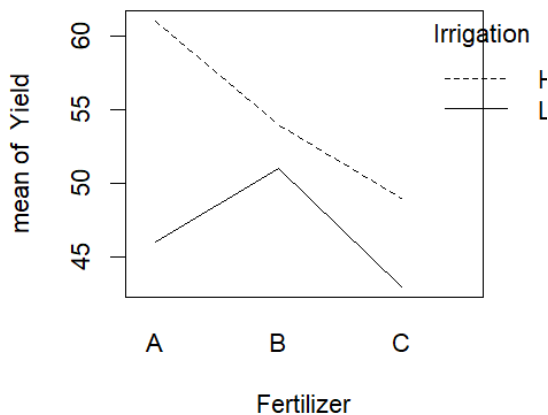
b. Interaction Effect: Fertilizer:Yield

- $p < 0.05 \rightarrow$ Significant interaction.
-  This means:
 -  Fertilizer **depends on** the Irrigation.
 -  You **cannot interpret main effects separately** — you must consider them **together**.

Visualizing the Interaction

```
with(data, interaction.plot(Fertilizer,Irrigation,Yield))
```

Result of plot:



Interpretation of the plot:

This interaction plot reveals a significant interaction between fertilizer type and irrigation level on crop yield.

- Fertilizer A performs best under high irrigation but poorly under low irrigation.
- Fertilizer B performs best under low irrigation and moderately well under high irrigation.
- Fertilizer C consistently results in lower yields compared to A and B, regardless of the irrigation level, and performs particularly poorly under low irrigation.

DAY 2

Statistic

Replication: Repetition of treatment. Estimate of experimental error. Generally 3 replication

Randomization: every treatment has an equal chance. Valid estimation of experimental error.

R Language

Local control: make the block in such way that the block make that each block is homogeneity and if compared to other is heterogeneity. Diminishing the experiment error.

Treatment: many

Factor: Irrigation, fertilizer

Levels: (high , low) and (a,b,c)

Factor * level = Treatment....

Single factor → those are just treatment.

And for the double factor...

Two factor → interaction..

1st factor : date of sowing(d1,d2,d3,d4,d5)

2nd factor: variety(v1,v2,v3,v4)

No of treatment here is (1st factor level * 2nd factor level)

There maybe a 3rd factorial .. And all is called factorial experiment....

Degree of freedom:

(Observation-1)

Variance calculation..variance here is experimental error.

Suppose here is 8 treatment(T1-T8)

Then the randomization, but here we only randomize the 7 and there is only one space for the T8 treatment.

So, there is no freedom for the T8

So the degree of freedom is (10-1)=9

Error degree of freedom.

R Language

Common design:

Completely randomize design(CRD) → no blocking

Experimental unit : homogeneous

Not used in field condition

Lab condition homogeneous, green house, glass house...

Merit:

1. Can use unequal number of replication.
2. Lost of replication is not so much of problem.

Demerit:

1. Any error in experimental unit error. So no variation should not be done.

ANOVA OF CRD

1. Difference between treatment
2. Difference within treatment. (this is called error) → from there we found the degree of freedom.

Randomized block Design(RBD) OR (RCBD)

Single factor

There may be level

Factorial RBD (FatRBD)

When 2 or more than 2 factor

Factor 1: Irrigation(high,low , medium)

Factor 2 : Fertilizer (A, B, C)

here we study interaction.

Split Plot Design (SPD)

R Language

1. Factor 1: tillage (Zero, RT, CT) → this is called main plot
2. Factor 2: N dose (100,150,200) → this is called sub plot

In case of factor 1 we must need a large plot here we cannot use the 5 m*3 m plot because the tractor cannot be run in this small area. So for the zero tillage we must need to big plot.

In this case accuracy can be low than the FatRBD.

Split Split Plot Design (double split plot design)

Factor 1: tillage → main plot

Factor 2: plant population → sub plot

Factor 3: N → sub sub plot

Factor 1 need big plot

Factor 2 need medium plot

Factor 3 need small plot

Strip plot design

When 2 plot need large plot then it is called the strip design

One is the horizontal factor and vertical factor.

Practice to bring the plot in the R

```
library(agricolae)
```

CRD analysis plot

```
crdplot <- design.crd(trt = c("T1", "T2", "T3"), r=4, seed = 123)
```

```
print(crdplot$book)
```

R Language

RBD analysis plot

```
RBDplot <- design.rcbd(trt= c("T1", "T2", "T3", "T4", "T5"), r=7, seed = 123)
```

```
print(RBDplot$book)
```

```
print(RBDplot$sketch)
```

15

DAY 3

```
data <- read.csv("RBD_sample_data.csv")
```

ANOVA model for RBD model

```
model <- aov(yield~treatment+block, data=data)
```

```
summary(model)
```

```
      Df Sum Sq Mean Sq F value    Pr(>F)
treatment  3  159.0    53.00     212 1.78e-06 ***
block       2   6.5     3.25       13 0.00659 **
Residuals   6   1.5     0.25
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Interpretation:

- treatment p-value is less than 0.05 so the treatment effect are significant
- block p-value is less than 0.05 that means the block difference also significant.

Model diagnostic

Normality test:

```
shapiro.test(residuals(model))
```

Interpretation: When the P-Value is more than 0.05 that means it's pass the normality test.

```
plot(model, which = 2)
```

which = 2 → Q-Q plot

Homogenicity test:

```
library(car)
```

R Language

```
leveneTest(yield~treatment, data = data)
```

P-Value more than 0.05 so pass the test.

Or,

```
plot(model, which = 1)
```

which = 1 → Residues vs Fitted plot

Post Hoc Test(LSD and HSD-Turkey) if the difference is significance.

Friedman test

When it is failed the diagnostics. So the Friedman test for the non parametric test. Like the Kruskal Wallis test. But Kruskal wallis is used for the no block structure.

```
nonndata <- read.csv("Friedman_RBD_sample_data.csv" )
```

```
modelnoon <- aov(yield~treatment+block, data = nonndata)
```

```
summary(modelnoon)
```

Diagnostic on normality and the homogeneity test. When the test is failed then we need to do friedman test, when there is a block.

Test code:

```
friedman.test(yield~treatment| block, data = nonndata)
```

then the post hoc for the non-normality data. Can use dunn test or the other available.

FatRBD MODEL

```
data <- read.csv("FatRBD_significant_example.csv" )
```

model creation

```
model <- aov(Yield~FactorA*FactorB+Block,data = data)
```

```
summary(model)
```

R Language

```
with(data, interaction.plot(FactorA, FactorB, Yield))
```

diagnosis:

```
shapiro.test(residuals(model))
```

```
library(car)
```

```
leveneTest(Yield~FactorA*FactorB, data = data)
```

When it's pass the all diagnosis we can perform post hoc.

DAY 4

```
data <- read.csv("split_plot_dataset.csv")
```

Split plot design

```
library(agricolae)
```

```
design <- design.split(trt1 = c("A1", "A2"),
```

```
                      trt2 = c("B1", "B2", "B3"),
```

```
                      r=2,
```

```
                      seed=123
```

```
)
```

design\$book → get the design

ANOVA model creation:

```
model <- aov(Yield~A*B+ Error(Rep/A), data=data)
```

```
summary(model)
```

Interpretation : For now trying to use the screenshot for the result.

R Language

```
Error: Rep
      Df Sum Sq Mean Sq F value Pr(>F)
Residuals 1   16.8    16.8

Error: Rep:A
      Df Sum Sq Mean Sq F value Pr(>F)
A       1   1.20    1.20   0.026  0.898
Residuals 1  45.63   45.63

Error: Within
      Df Sum Sq Mean Sq F value Pr(>F)
B       2  31.24   15.62   1.233  0.383
A:B     2  73.97   36.99   2.920  0.165
Residuals 4  50.67   12.67
```

DAY 5

```
data <- read.csv("two_way_anova_data.csv")
```

```
model <- aov(Yield~ Treatment*Irrigation, data = data)
```

```
summary(model)
```

```
plot(model)
```

```
shapiro.test(residuals(model))
```

```
library(car)
```

```
leveneTest(Yield~Treatment*Irrigation, data = data)
```

Post-Hoc test with emmeans

```
library(emmeans)
```

```
#compare Treatment level or yield within each irrigation level
```

```
emmeans(model, pairwise~ Treatment | Irrigation)
```


R Language

LSD or Turkey style post hoc test

LSD style :-

```
emmeans(model, pairwise~ Treatment | Irrigation, adjust = "none")
```

HSD style :-

```
emmeans(model, pairwise~Treatment | Irrigation, adjust= "tukey")
```

Compare all this with interaction

```
emmeans(model, pairwise~Treatment*Irrigation)
```

```
#add group letters
```

```
#for main effect
```

```
library(emmeans)
```

```
library(multcompView)
```

```
cld(emmeans(model, ~ Treatment | Irrigation, Letters=letters))
```

```
#l and chatgpt are failed to add letters in this emmeans. Maybe because of the package didnot load fully.. will try later.
```

16

DAY 6 +7

CRD design analysis

1. Non-normal data

model creation:

```
datacrd <- read.csv("crd_data.csv")
```

```
modelcrd <- aov(yield~treatment, data= datacrd)
```

then,

diagnoses first to check the normality of dataset. If it's fail to the test then we need to do build a model for non normal data and Post hoc test next. Kruskal - Wallis test for non normality data, as the plot has no block. If it's has block we need to do Freidman test.

R Language

Model building:

```
kruskal.test(yield~treatment, data= datacrd)
```

Post Hoc:

```
library(FSA)
```

```
dunnTest(yield~treatment, data=datacrd, method= "bonferroni")
```

2. Normally distributed data

Model building:

```
dtcrdnor <- read.csv("crd_normal_data.csv")
```

```
mdlcrdnor <- aov(yield~treatment,data=dtcrdnor)
```

Then do the diagnosis and it will pass the test as the data is normally distributed.

```
summary(mdlcrdnor)
```

If the P-Value is significance do the Post Hoc.

Post hoc with the emmeans:

- a. LSD + compact letter display

```
library(emmeans)
```

```
library(multcomp)
```

```
library(multcompView)
```

```
multcomp::cld(emmeans(mdlcrdnor,~treatment), adjust= "none",Letters=letters)
```

- b. HSD + compact letter display

```
multcomp::cld(emmeans(mdlcrdnor,~treatment), adjust= "tukey",Letters=letters)
```

RBD model analysis

Anova model on RBD

R Language

1. Non normal data

```
datarbd <- read.csv("rbd_data.csv")
```

model building:

```
mdlrbd <- aov(yield~treatment+block,data=datarbd)
```

diagnosis for the dataset to see the status of the data. (normality test and homogeneity test). If it is failed need to create a model with the Friedman test. And then post hoc with another thing. I will learn about it latter.

2. Normally distributed data

```
dtrbdnor <- read.csv("rbd_normal_data.csv")
```

model building:

```
mldrbdnor <- aov(yield~treatment +block, data=dtrbdnor)
```

diagnosis the dataset.

```
summary(mldrbdnor)
```

If the P-Value is less than 0.05 then need to post hoc of the dataset.

Post Hoc:

a. LSD test

```
library(emmeans)
```

```
library(multcomp)
```

```
library(multcompView)
```

```
multcomp::cld(emmeans(mldrbdnor, ~ treatment), adjust = "none", Letters = letters)
```

b. HSD test

NB: must call the library (emmeans, multcomp, multcompView) first.

```
multcomp::cld(emmeans(mldrbdnor, ~ treatment), adjust = "tukey", Letters = letters)
```

R Language

In this case we don't use "|". Because the block is used to reduce the error. When we have another factor then we can use this symbol. When there is an interaction between these then we can use "*" to get accurate result. So, the post hoc is similar as the CRD.

lmer model on RBD

block has the random effect not the fixed effect. That's why it's wise to use lmerTest.

```
library(lme4)
```

```
library(lmerTest)
```

lmer model building:

```
lmerbd <- lmer(yield~treatment+(1|block), data=dtrbdnord)
```

```
summary(lmerbd)
```

anova(lmerbd, type=3) → Give anova like result.

Interpretation:

See the treatment T1 and T2 P-Value. It creates a baseline (T1) then give the result.

Post Hoc:

```
multcomp::cld(emmeans(lmerbd, ~ treatment), adjust = "tukey", Letters = letters)
```

FatRBD model analysis

Anova model on FatRBD

```
dtfatrbd <- read.csv("factorial_rbd_normal.csv")
```

model building:

R Language

```
mdlfatrbd <- aov(Yield~Fertilizer*Variety+Block , data= dtfatrbd)
```

Then perform all the diogonsis. When it is pass the test.

```
summary(mdlfatrbd)
```

If it is passed the hypotheses means the P-Value is less than 0.05 then perform the post hoc.

Post Hoc:

```
Library(emmeans)
```

```
library(multcomp)
```

```
library(multcompView)
```

a. Compare with the interaction

```
multcomp::cld(pairs(emmeans(mdlfatrbd, ~Fertilizer*Variety), adjust = "tukey"),Letters=letters)
```

It is difficult to analysis.

b. Compare fertilizer within each variety

```
multcomp::cld(pairs(emmeans(mdlfatrbd, ~Fertilizer| Variety), adjust="tukey"),Letters= letters)
```

c. Compare variety within each fertilizer

```
multcomp::cld(pairs(emmeans(mdlfatrbd, ~Variety| Fertilizer),adjust="tukey"),Letters=letters)
```

Imer model on FatRBD

model building:

```
lemrfatrbd <- lmer(Yield~Fertilizer*Variety + (1| Block), data=dtfatrbd)
```

```
summary(lemrfatrbd)
```

anova(lemrfatrbd, type=3) → give Anova type result

R Language

NB: post hoc is similar as the ANOVA model before

Split Plot design analysis

```
dtsplit <- read.csv("split_plot_data.csv")
```

1. Anova model creation:

In this case ANOVA will not be efficient although make one.

```
mdl_spl_anova <- aov(yield~irrigation*fertilizer+ Error(block/irrigation),data=dtsplit)

summary(mdl_spl_anova)
```

2. lmer model build:

```
library(lme4)
```

```
library(lmerTest)
```

model build:

```
mdl_spl_lmer <- lmer(yield~irrigation*fertilizer +(1|block/irrigation),data=dtsplit)
```

```
summary(mdl_spl_lmer)
```

```
anova(mdl_spl_lmer, type=3) → just give the ANOVA type result.
```

Post Hoc:

1. single factor

a) Turkey test for the irrigation

```
library(emmeans)
```

```
emmeans(mdl_spl_lmer,pairwise~irrigation,adjust="tukey")
```

b) Turkey test for fertilizer + the letters

```
library(multcomp)
```

```
multcomp::cld(emmeans(mdl_spl_lmer,pairwise~fertilizer),adjust="tukey",Letters=letters)
```

R Language

2. Double factor

- a. With the all two interaction

```
emmeans mdlSplLmer, pairwise ~ irrigation * fertilizer, adjust = "tukey")
```

difficult to analysis

- b. Test for the fertilizer on different irrigation

```
emmeans mdlSplLmer, pairwise ~ fertilizer | irrigation, adjust = "tukey")
```

- c. Test for the irrigation effect with different fertilizer+ add the letters

```
multcomp::cld(emmeans mdlSplLmer, pairwise ~ irrigation | fertilizer), adjust = "tukey", Letters = letters)
```

NB: It's didn't give me the pairwise result. The `lm()` and the `aov()` gives the pairwise result and the group together but this(`lmer()`) donot give that.

- d. For the pairwise result

```
emmeans mdlSplLmer, pairwise ~ irrigation | fertilizer, adjust = "tukey")
```

Theory

1. Fixed vs Random Effects

- **Fixed effect:** Treatments you are testing (e.g., fertilizer).
- **Random effect:** Variation sources not of main interest (e.g., blocks, years).

2. `aov()` Function

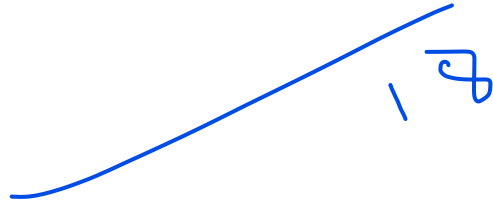
- Use when **all factors are fixed** (e.g., CRD).
- Assumptions:
 - Normality of residuals

R Language

- Homogeneity of variance

3. Mixed Models with lmer()

- Use when you have **random effects** (e.g., RBD, split-plot).
- Doesn't give p-values by default → use lmerTest package.



3 Week

DAY 1

Regression

Regression is a statistical method to model and analyze the relationship between a **dependent variable** (response) and one or more **independent variables** (predictors).

```
data <- read.csv("agriculture_yield_data.csv")
```

model building:

```
model <- lm(yield~nitrogen, data=data)
```

```
model2 <- lm(yield~nitrogen+rainfall, data=data)
```

For better interpretation yield and soil nitrogen in kg/ha and rainfall in mm

```
summary(model)
```

```
summary(model2)
```

interpretation of summary:



Model Summary:

```
lm(formula = yield ~ nitrogen + rainfall, data = data)
```

This model tries to **predict crop yield** using two predictors: **nitrogen** and **rainfall**.

1. Coefficients Table (Effect of Each Variable)

R Language

Term	Estimate	Std. Error	t-value	p-value	Meaning
Intercept	-0.39660	0.66741	-0.594	0.571	When nitrogen = 0 & rainfall = 0 → predicted yield = -0.3966 kg/ha → not realistic (used for calculation only)
Nitrogen	0.03782	0.03499	1.081	0.316	For every 1 unit ↑ nitrogen → yield ↑ 0.03782 kg/ha (not statistically significant)
Rainfall	0.01926	0.02175	0.885	0.405	For every 1 mm ↑ rainfall → yield ↑ 0.01926 kg/ha (not statistically significant)

★ How to interpret p-values:

- If $p < 0.05$ → statistically significant (important)
- Here, **none of the variables are statistically significant** individually.

2. Residuals

Min 1Q Median 3Q Max

-0.3651 -0.0139 0.0257 0.0769 0.2447

- These show how far the predicted yield is from the actual yield.
- **Smaller values** = better prediction
- **Symmetric and small spread** = good sign (no major outliers)

3. Residual Standard Error

Residual standard error: 0.1915

- Average distance between actual and predicted yield
- Lower = better
- Here: prediction error ~0.19 kg/ha → **quite small = good fit**

4. Multiple R-squared

Multiple R-squared: 0.9518

- 95.18% of yield variation is explained by **nitrogen + rainfall**
- **Very strong model fit**
- But this value can **increase even with useless variables**

R Language

5. Adjusted R-squared

Adjusted R-squared: 0.938

- Adjusts for number of variables
- Still very high (93.8%) → model is **realistically strong**
- Better measure than R^2 when comparing multiple models

6. F-statistic

F-statistic: 69.12 on 2 and 7 DF, p-value: 2.458e-05

- Tests if **nitrogen and rainfall together** explain the variation in yield
- High F (69.12) + $p < 0.001$ → **model is significant overall**
- Even if individual variables not significant, model **together is powerful**

7. Final Evaluation (What's Good)

Metric	This Model	Is it Good?
R-squared	0.9518	✓ Yes (strong fit)
Adjusted R-squared	0.9380	✓ Yes (realistic fit)
Residual std. error	0.1915	✓ Small → good model
F-statistic p-value	0.000024	✓ Very significant
Variable p-values	> 0.05	✗ Not significant alone

8. How R^2 Relates to Prediction

- $R^2 = 0.95$ means model **explains 95% of the yield variation**.
- Not "95% accurate" — it doesn't guarantee future prediction accuracy if important factors are missing.

Actual = real observed value (what you **actually measured** in your experiment).

- Example: A field gave 3.5 tons of yield → this is the **actual** yield.

Predicted = value that your **model estimated** using nitrogen, rainfall, etc.

R Language

- Example: Model predicted 3.2 tons → this is the **predicted** yield.

Mean = average of all actual yields.

- Add all actual yields and divide by total number of observations.
- It's like the “center” of the yield values.

◆ 3 Types of Variation:

Type	Formula	Meaning
Total variation	$SST = \sum(\text{Actual} - \text{Mean})^2$	How much yield varies overall
Explained variation	$SSR = \sum(\text{Predicted} - \text{Mean})^2$	How much model explains
Unexplained variation	$SSE = \sum(\text{Actual} - \text{Predicted})^2$	Error or residuals left by model

Residual = actual – predicted

◆ How R² is Related:

$R^2 = \text{Explained variation} / \text{Total variation}$

$= SSR / SST$

So:

- If $R^2 = 0.95 \rightarrow$ **95% of the total variation in yield** is explained by nitrogen and rainfall
- The remaining 5% is due to error (unexplained)

9. Overfitting

- When the model uses **too many variables** including those not really important.
- It performs well on current data but poorly on new data.

🚩 Why R² alone can be misleading:

- High R² could mean **overfitting**.
- That's why **Adjusted R²** is preferred — it lowers when unnecessary variables are added.

10. What if Other Important Factors Are Missing?

- Example: **Irrigation** might be important but not included.

R Language

- Model assumes: conditions like irrigation are **constant or optimal**.
- If not true, predictions may be biased.

DAY 2

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 3\\Week 3 materials\\Day 2")
```

```
list.files()
```

```
data <- read.csv("yield_soil_data.csv")
```

```
library(ggplot2)
```

```
ggplot(data, aes(nitrogen, yield)) +  
  geom_point(color = "blue", size = 3) +  
  geom_smooth(method = "lm", se = TRUE, color = "red") +  
  labs(title = "Nitrogen vs Yield", x = "Soil Nitrogen", y = "Yield") +  
  theme_minimal()
```

- Linear Trend? → If points follow a straight line pattern, it's a linear relationship
- Increasing Yield? → If the line goes up, it means more nitrogen = higher yield.
- Good Fit? → If most points are close to the red line, the fit is strong.

```
ggplot(data, aes(rainfall, yield)) +  
  geom_point(color = "green", size = 3) +  
  geom_smooth(method = "lm", se = TRUE, color = "darkgreen") +  
  labs(title = "Rainfall vs Yield", x = "Rainfall (mm)", y = "Yield") +  
  theme_minimal()
```

- can see that the yield is increasing with the rainfall
- if the line is flat, rainfall might not impact yield much. If the line is steep, then rainfall has a stronger effect

R Language

- se= TRUE shows a shaded area.

DAY 3

Correlation Matrix

- Shows how **strongly** variables are related to each other.
- Correlation values range:
 - +1 → perfect positive correlation → perfect increase-increase relation
 - 0 → no correlation → no clear relationship
 - -1 → perfect negative correlation → one increases, other decreases

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 3\\Week 3 materials\\Day 3")
```

```
list.files()
```

```
data <- read.csv("high_correlation_agriculture.csv")
```

```
cor(data)
```

```
#give the correlation for only selected variables
```

```
cor(data[,c("yield", "rainfall")])
```

pairs.panels()

See all the variable relationship at once. and ggplot is seeing the one relationship in details..

```
install.packages("psych")
```

```
library(psych)
```

```
pairs.panels(data,
```

```
  method="pearson",
```

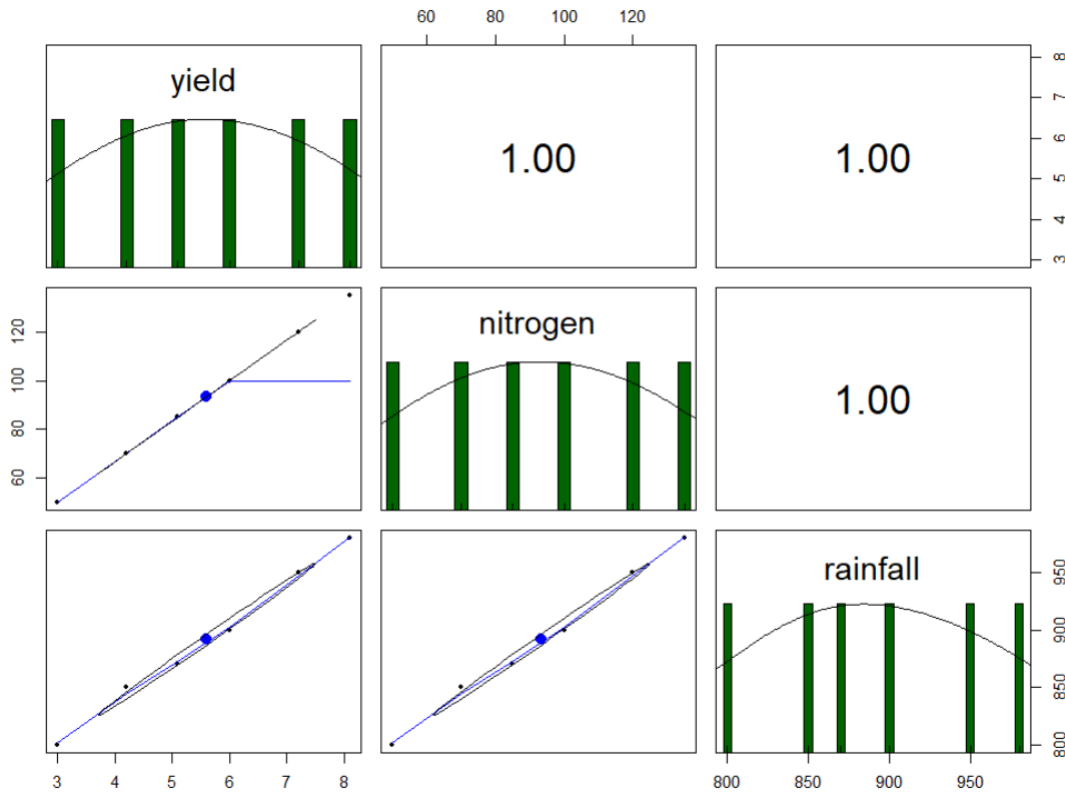
```
  hist.col="darkgreen",
```

```
  desity=TRUE,
```

```
  ellipses = TRUE
```

```
)
```

R Language



Upper Triangle – Correlation Coefficients:

- Shows the **strength and direction** of relationship between variables.
- Values range from:
 - +1 → perfect positive
 - 0 → no correlation
 - 1 → perfect negative
- Example:
 - 0.98 between yield and nitrogen → very strong positive relationship.
- If stars=TRUE, significance levels are marked (e.g., *, **, ***).

Diagonal – Histograms (with density lines if density=TRUE):

- Shows **distribution** of each variable.
- Use it to check:
 - Normality (bell-shaped)
 - Skewness (leaning left or right)
 - Outliers (isolated bars)

R Language

Lower Triangle – Scatter Plots (with ellipses if ellipses=TRUE):

- Visualizes the **trend between pairs of variables**.
- Line-shaped or elliptical = strong linear relation.
- Wide or spread = weak or no correlation.
- Outliers appear as distant points.

Ellipses:

- Thinner = stronger linear relationship.
- Round = weak or no relationship.
- Direction shows positive or negative trend.

VIF

- It measures **how much one predictor variable can be predicted by other predictors**.
- The higher the VIF, the more redundant (overlapping) info that predictor has with others.

```
library(car)
```

```
model <- lm(yield~nitrogen+rainfall, data=data)
```

```
vif(model)
```

Meaning of VIF values:

VIF Value	Interpretation
1	No correlation with other variables (ideal)
1 to 5	Moderate correlation, usually okay
> 5	High correlation, possible problem
> 10	Serious multicollinearity, bad for regression

If I have high multicollinearity so the next step is how to manage it...

R Language

Managing Multicollinearity

1. collect more data

2. remove one variable

```
model1 <- lm(yield~nitrogen, data= data)
```

Combine variables into one

```
data$combined <- (data$nitrogen+data$rainfall)/2
```

```
model2 <- lm(yield~combined, data=data)
```

3. PCA (Principal Component Analysis)

```
library(stats)
```

```
#standardize the variables
```

```
vars <- scale(data[,c("nitrogen", "rainfall")])
```

```
#run PCA
```

```
pca_result <- prcomp(vars)
```

```
summary(pca_result)
```

#here the PC1 proportion of variance is .99 so that means it is very great pca. we can just use this one. Must be more than 70 percent.

```
#creating a model
```

```
modelpca <- lm(yield~pca_result$x[,1], data=data)
```

```
summary(modelpca)
```

Theory

◆ Step 1: Load stats library

R Language

```
library(stats)
```

- It contains `prcomp()` — the function to perform PCA.

◆ Step 2: Standardize the variables

```
vars <- scale(mydata[, c("nitrogen", "rainfall", "temperature")])
```

✅ What does this do?

- Picks only **3 columns**: nitrogen, rainfall, temperature.
- `scale()` standardizes them:
 - **Subtracts the mean**
 - **Divides by standard deviation**

🧠 Why standardize?

- PCA is affected by **scale** of variables.
 - E.g., rainfall = 100s, nitrogen = 10s
 - Bigger values can dominate PCA.
- Standardizing gives **equal weight** to all.

◆ Step 3: Run PCA

```
pca_result <- prcomp(vars)
```

✅ What does this do?

- Performs **Principal Component Analysis** on vars.
- Returns:
 - **PC1, PC2, PC3** (new uncorrelated variables)
 - How much **variance** each PC explains
 - **Rotation matrix** (loadings for each original variable)

📌 Example output:

```
summary(pca_result)
```

Might give:

R Language

PC	Proportion of Variance
PC1	0.85
PC2	0.12
PC3	0.03

👉 This means:

- PC1 alone explains **85% of the variation** in the data.
- So PC1 is a **very good summary** of nitrogen, rainfall, and temperature.

◆ Step 4: Use PC1 in regression

```
model <- lm(yield ~ pca_result$x[,1], data = mydata)
```

✅ What's happening here?

- `pca_result$x[,1]` is the **score of PC1** for each row.

🧠 Think of it like this:

- Instead of using 3 variables (nitrogen, rainfall, temp), you're using **just 1 new variable (PC1)**.
- That PC1 is a **summary** of the original three.
- It improves **model stability**, removes **multicollinearity**, and simplifies analysis.

Final tip:

- Aim to explain **≥80% variance with as few PCs as possible**
- Start simple (PC1), add more PCs only if accuracy or model fit is poor
- Always check **cumulative variance** in PCA summary before deciding

4. Ridge Regression:

- Ridge Regression is an **improved version of linear regression**.
- It adds a **penalty term** (called **L2 penalty**) to the regression equation.
- The goal is to **shrink large coefficients** — this helps when:
 - Variables are **correlated** (multicollinearity)
 - There are **too many predictors** (risk of overfitting)

R Language

- Keeps **all variables** in the model (doesn't remove any)
- Controls and **stabilizes coefficient** (shows how strongly a predictor affects the target) values
- Helps build **robust models** with better generalization

What's the Penalty in Ridge Regression?

In **normal linear regression**, the goal is:

Minimize $SSE = \sum (\text{Actual} - \text{Predicted})^2$

In **Ridge Regression**, we **add a penalty**:

Minimize: $SSE + \lambda \times \sum (\beta_1^2 + \beta_2^2 + \dots + \beta_n^2)$

- **SSE** = sum of squared errors (how wrong your model is)
- $\beta_1, \beta_2, \dots, \beta_n$ = model coefficients (weights for each variable)
- λ (**lambda**) = penalty strength

Why Shrinking Helps?

🌟 Problem with Large Coefficients:

- The model reacts **too strongly** to small changes in data.
- That leads to **overfitting** — the model fits the noise, not just the pattern.

✅ Solution with Ridge:

- Ridge shrinks the β values.
- Smaller $\beta \rightarrow$ **less sensitivity to noise** $\rightarrow$ **simpler, more stable model**

Lambda (λ) — The Tuning Key

🧪 What Happens if:

- $\lambda = 0 \rightarrow$ no penalty $\rightarrow$ same as normal regression
- $\lambda = \text{small (e.g., 0.01)} \rightarrow$ very light shrinking
- $\lambda = \text{large (e.g., 100)} \rightarrow$ very strong shrinking (coefficients close to 0)

R Language

λ Value	Effect	Coefficients
$\lambda = 0$	No penalty, normal regression	Original β
Small λ	Light penalty	Slightly shrunk β
Medium λ	Balanced penalty (good performance)	Well-shrunk β
Large λ	Strong penalty	All β close to 0

Code in R:

```
library(glmnet)
```

```
data1 <- read.csv("agricultural_yield_data.csv")
```

```
#Prepare predictor matrix and response vector
```

```
a <- as.matrix(data1[,c("Soil_Nitrogen","Soil_Phosphorus","Rainfall","Temperature")])
```

- Extracts the predictor variables from mydata and converts them into a **matrix** called x.
(glmnet requires predictors in matrix form.)

```
b <- data1$Yield
```

```
#Fit ridge regression model with cross-validation to find best lambda
```

```
ridge_model <- cv.glmnet(a,b,alpha=0)
```

- Fits Ridge Regression model to the data using **cross-validation** (cv.glmnet).
- $\alpha = 0$ tells glmnet to perform **Ridge Regression** (if $\alpha=1$, it's Lasso).
- Cross-validation automatically splits data, fits models with different lambda values (penalties), and selects the best lambda that minimizes prediction error.

```
#extract the best lambda value
```

```
best_lambda <- ridge_model$lambda.min
```

```
best_lambda
```

R Language

```
#predict yield using the ridge model and best lambda
```

```
Predicted_yield <- predict(ridge_model, s=best_lambda, newx=a)
```

```
Predicted_yield
```

DAY 4

Already done

DAY 5

Assapmtion

1. Independenc e
2. Linearity → residuals vs fitted. Need to be scattered
3. Normality → qq plot. The point is near the line.
4. Constant variance → scale-location

Simple Linear Regression: Diagnostics (part 4 of 4)

Assumption	Formal definition www.youtube.com	Interpretation	Alternatives
Independence	$\epsilon_i, \dots, \epsilon_n$ are iid Errors are independent and identically distributed.	Observations come from distinct experimental units.	Explicitly model the source of dependence (e.g., mixed models).
Linearity	$y = \beta_0 + \beta_1 x + \epsilon$ The relationship can be expressed in terms of intercept and slope.	Increasing x by 1 is associated with a fixed change in y .	Transformation, splines.
Normality	$\epsilon \sim \mathcal{N}(0, \sigma^2)$ The error follows a normal distribution.	Observations are normally distributed along the regression line.	Regression based on a different probability distribution (e.g., GLM), bootstrapping.
Constant variance	σ^2 is constant The error has the same variance everywhere along the line.	Variance is constant along the regression line.	Robust standard errors, bootstrapping.
No influential outliers	-	Though not really an assumption, you will want to assess the validity of highly influential data points.	Robust standard errors, removal under specific conditions (e.g., incorrect data entry).

11:49 / 12:00 • Summary >

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 3\\Week 3 materials\\Day 5")
```

R Language

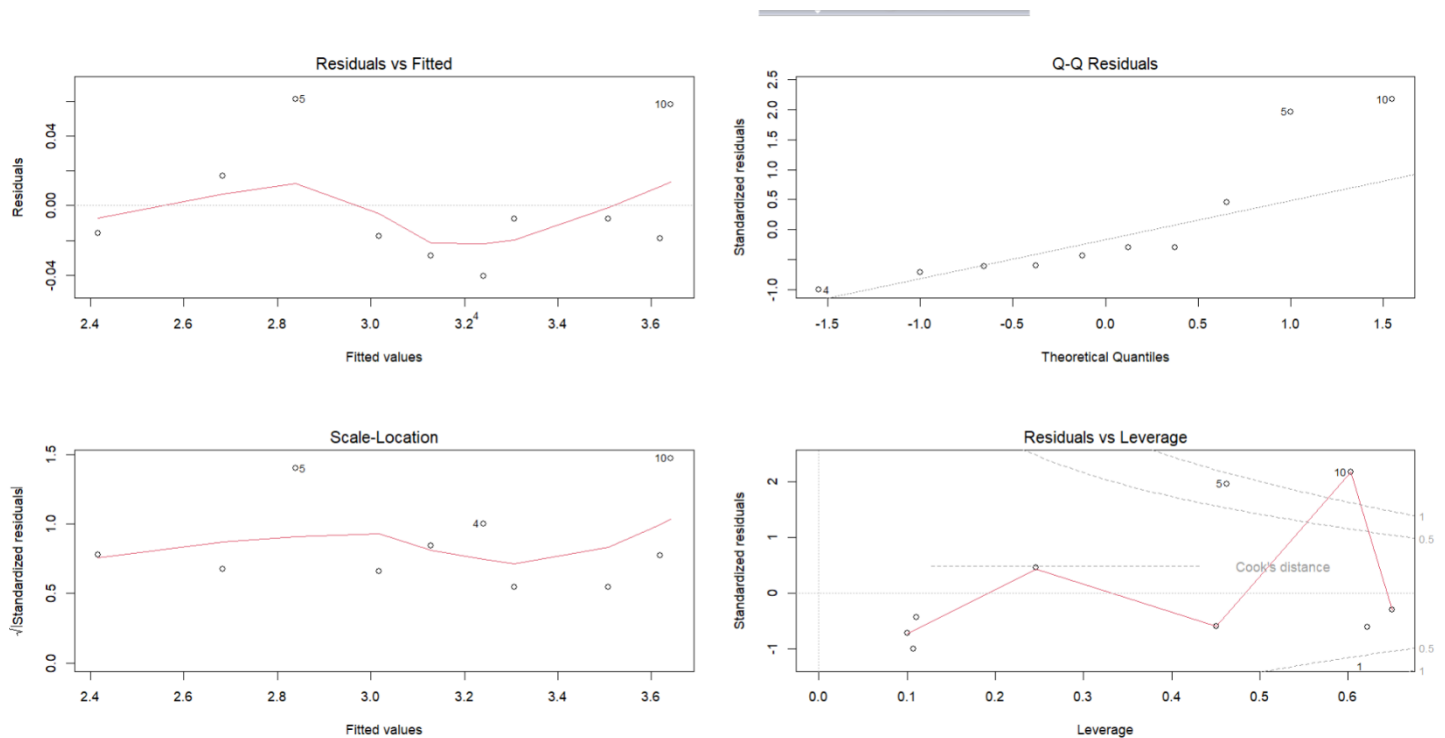
```
list.files()
```

```
data <- read.csv("agri_yield_data (1).csv")
```

```
model <- lm(Crop_Yield~Fertilizer_kg+Rainfall_mm+Soil_Nitrogen, data=data)
```

```
par(mfrow=c(2,2))
```

```
plot(model)
```



Explanation and interpretation:

1 Residuals vs Fitted

- **What to check:** Should be random/no clear pattern.
- **Bad sign:** Curved shape or funnel → non-linearity or heteroscedasticity.

2 Normal Q-Q

- **What to check:** Points should follow the straight line.
- **Bad sign:** Deviations at the ends → residuals not normal.

3 Scale-Location

R Language

- **What to check:** Horizontal spread.
- **Bad sign:** Increasing or decreasing spread → variance is not constant.

Residuals vs Leverage

- **What to check:** Outliers or influential points.
- **Bad sign:** Points outside Cook's distance → highly influential.

```
library(car)
```

```
qqPlot(model)
```

```
ncvTest(model)
```

 qqPlot():

- Points near the line = residuals are normal.
- Far from line = skewness/heavy tails.

 ncvTest():

- Null hypothesis: **residual variance is constant**
- p-value > 0.05 →  homoscedasticity
- p-value < 0.05 →  heteroscedasticity

need to know about the plot more deeply!!!what's the real meaning is actually.

DAY 6

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 3\\Week 3 materials\\Day 6")
```

```
list.files()
```

```
data <- read.csv("day6_crop_data.csv")
```

R Language

```
model1 <- lm(yield ~ nitrogen + rainfall, data = data)
```

```
model2 <- lm(yield ~ nitrogen + rainfall + temperature, data = data)
```

```
model3 <- lm(yield ~ nitrogen + rainfall + temperature + pH + organic_matter, data = data)
```

```
AIC(model1, model2, model3)
```

```
BIC(model1, model2, model3)
```

```
#Lowest AIC for best prediction
```

```
#Lowest BIC for simpler model
```

```
#stepwise regression
```

```
full_model <- lm(yield ~ nitrogen + rainfall + temperature + pH + organic_matter, data = data)
```

```
step_model <- step(full_model, direction = "both")
```

```
summary(step_model)
```

theory:

1. AIC (Akaike Information Criterion)

- Measures how well a model fits the data while penalizing complexity (number of variables).
- Formula balances **goodness of fit** and **model simplicity**.
- **Lower AIC** means a better model.
- Useful to compare different models and avoid overfitting.

2. BIC (Bayesian Information Criterion)

- Similar to AIC but penalizes complexity **more strictly**.
- Prefers simpler models, especially when the sample size is large.
- **Lower BIC** means a better, simpler model.

3. Stepwise Regression

- A method to **automatically select the best combination of variables**.
- Uses AIC to add or remove variables step-by-step.

R Language

- Types:
 - **Forward selection:** starts with no variables, adds one at a time.
 - **Backward elimination:** starts with all variables, removes one at a time.
 - **Stepwise (both):** adds/removes variables in both directions to optimize model.

4. Choosing the Best Model

- Fit multiple candidate models with different variables.
- Compare their AIC and BIC values:
 - Pick the model with the **lowest AIC** if prediction accuracy is priority.
 - Pick the model with the **lowest BIC** if simplicity and interpretability matter more.
- Use stepwise regression to help select variables automatically.
- Always consider adjusted R^2 and residual diagnostics to confirm model quality.

DAY 7

RMSE (Root Mean Square Error)

RMSE (Root Mean Square Error) is a metric that tells you **how much error your model makes when predicting**.

It shows the **average difference** between actual and predicted values in the **same unit** as your output (e.g., tons, kg).

RMSE Formula:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_{\text{actual}} - y_{\text{predicted}})^2}$$

- **y_actual** = observed values
- **y_predicted** = predicted values from the model
- **n** = number of observations

Key Points:

- **Lower RMSE = better model prediction**
- **Takes square root** to return to the original unit
- **Sensitive to large errors** (big errors are penalized more)

R Language

$rRMSE = (RMSE / \text{mean of actual yields}) \times 100$

🔍 ✅ $rRMSE < 10\%$ = Good

🔍 ⚠️ $rRMSE 10\text{--}20\%$ = Moderate

🔍 ❌ $rRMSE > 20\%$ = Poor

📊 RSE vs RMSE:

Metric	Meaning	Use
RSE	Error within training data	For model diagnostics
RMSE	Error on new/test data	For prediction accuracy ✅

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 3\\Week 3 materials\\Day 7")
```

```
list.files()
```

```
data <- read.csv("yield_data.csv")
```

```
models <- lm(yield~nitrogen, data=data)
```

```
modelm <- lm(yield~nitrogen+rainfall, data=data)
```

```
#rmse test
```

```
predicted <- predict(modelm)
```

```
actual <- data$yield
```

```
rmse <- sqrt(mean(actual-predicted)^2)
```

R Language

```
rmse
```

```
#here the RMSE is 1.509838e-15. means average prediction error is 1.509838e-15 units.(eg., tons per hectare)
```

```
summary(models)
```

```
summary(modelm)
```

```
# here the r squared is 0.4 is low so that the model isn't fit for prediction
```

```
data1 <- read.csv("yield_data_high_r2.csv")
```

```
model2 <- lm(yield~nitrogen+rainfall, data=data1)
```

```
summary(model2)
```

```
#predicted yield for the new values
```

```
new_data <- data.frame(nitrogen= c(90,100), rainfall=c(120,130))
```

```
predicted_yield <- predict(model2, newdata = new_data)
```

```
predicted_yield
```

```
#diagnosis of the model
```

```
par(mfrow=c(2,2))
```

```
plot(model2)
```

```
library(car)
```

```
qqPlot(model2)
```

```
#point near the line
```

R Language

```
ncvTest(model2)
```

#p value is 0.8 which is more than 0.05, that means homoscedasticity

#RMSE test

```
actual_yield <- data1$yield
```

```
rmse1 <- sqrt(mean(actual_yield-predicted_yield)^2)
```

```
rmse1
```

#here the RMSE is 0.73. means average prediction error is 0.73 units.(eg., tons per hectare)

#sqrt means square root.

Report for the model

📌 Regression Model Used

```
model2 <- lm(yield~nitrogen+rainfall, data=data1)
```

📌 Summary of the Model

- $R^2 = 0.55 \rightarrow$ moderately good fit
- Nitrogen coefficient = 0.035 (p = 0.0005, very significant)
- Rainfall coefficient = 0.034 (p = 0.00003, very significant)
- _____

📌 Prediction for New Data

Nitrogen	Rainfall	Predicted Yield
90	120	9.65
100	130	10.34

📌 Model Diagnostics

- ☒ Residuals are roughly **normally distributed** (QQ plot looks fine)

R Language

-  No clear pattern in **residuals vs fitted** plot → **Homoscedasticity** present
 - RMSE is 0.73
-

Conclusion

- The model **fits well** and predicts yield from nitrogen and rainfall accurately.
- This can be **useful for precision agriculture**, especially in **fertilizer planning** and **yield forecasting**.

Week =3+1

DAY 1

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 4\\Materials\\Day 1")
```

```
list.files()
```

```
install.packages("AgroR")
```

```
library(AgroR)
```

```
#list all the example datasets included in the package
```

```
data(package="AgroR")
```

```
data("enxofre")
```

```
head(enxofre)
```

```
#agroR documentation
```

```
ls("package:AgroR")
```

```
?DIC
```

R Language

✓ 4. agroR Key Input Terms

Input	Meaning	Used In
trat	Single treatment factor	CRD, RBD
resp	Response variable	All designs
bloco	Block/Replication	RBD
f1	First factor in factorial design	2-way factorial
f2	Second factor in factorial	2-way factorial

✓ 5. agroR Core Functions (Basic Overview)

Function	Use Case	Design Type
CRD()	ANOVA + plot for one treatment	Completely Random
DIC()	ANOVA + plot with blocks	Randomized Block
fat2()	Two-factor factorial ANOVA	Factorial Design
pol()	Polynomial regression	Dose-response
mcomp()	Mean comparison (Tukey, LSD etc.)	All designs

DAY 2

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 4\\Materials\\Day 2")
```

```
list.files()
```

```
datacrd <- read.csv("CRD_sample_dataset.csv")
```

```
datacrd
```

```
library(AgroR)
```

R Language

```
with(datacrd,DIC(trat,resp,addmean=TRUE,ylab="resp",fill="trat",quali=TRUE))
```

✅ Use quali = TRUE when:

- Your treatment is **not a number**, but a **name or type**
- Examples:
 - Fertilizer A, B, C
 - Variety 1, Variety 2
 - Control, Treated
- You want to **compare means** using letters (a, b, ab)

💡 Think: "**Types or Categories**" → TRUE

❌ Use quali = FALSE when:

- Your treatment is a **number**
- Examples:
 - 0 kg, 50 kg, 100 kg fertilizer
 - 10°C, 20°C, 30°C
- You want to see a **trend** (line graph or curve) or fit a **regression**

💡 Think: "**Numbers or doses**" → FALSE

🧪 Example:

Treatment	quali	Why?
A, B, C	TRUE	These are categories
0, 50, 100 kg	FALSE	These are numeric levels

Interpretation of result:

1. Normality of Errors (Shapiro-Wilk test)

- **p-value = 0.935 > 0.05**
- Errors are **normally distributed** → Good for ANOVA assumptions

R Language

2. Homogeneity of Variances (Bartlett test)

- **p-value = 0.521 > 0.05**
 - Variances are **equal across treatments** → Another ANOVA assumption met
-

3. Independence of Errors (Durbin-Watson test)

- **p-value = 0.792 > 0.05**
- Errors are **independent** → No autocorrelation problem

Independence of Errors

- Errors (differences between observed and predicted values) should be **independent**, meaning:
 - The error for one observation doesn't affect another.
 - No pattern or correlation between residuals.
- **Why important?**
If errors are correlated (dependent), it violates ANOVA assumptions, making results unreliable.
- **Test used:** Durbin-Watson test checks if errors are independent.

Simple analogy:

- **Independence of errors:** Each mistake/error is random and doesn't influence other mistakes.
-

4. Additional Info

- **CV = 2.18%** → Very low variability → Good experiment precision
 - **Mean response = 20.81** (average value of all treatments)
 - **No outliers** → Data is clean
-

5. ANOVA Table

Source	Df	Sum Sq	Mean Sq	F value	Pr(F)
--------	----	--------	---------	---------	-------

Treatment	3	36.28	12.09	58.76	0.00000859 < 0.05 (significant)
-----------	---	-------	-------	-------	---------------------------------

R Language

Source Df Sum Sq Mean Sq F value Pr(F)

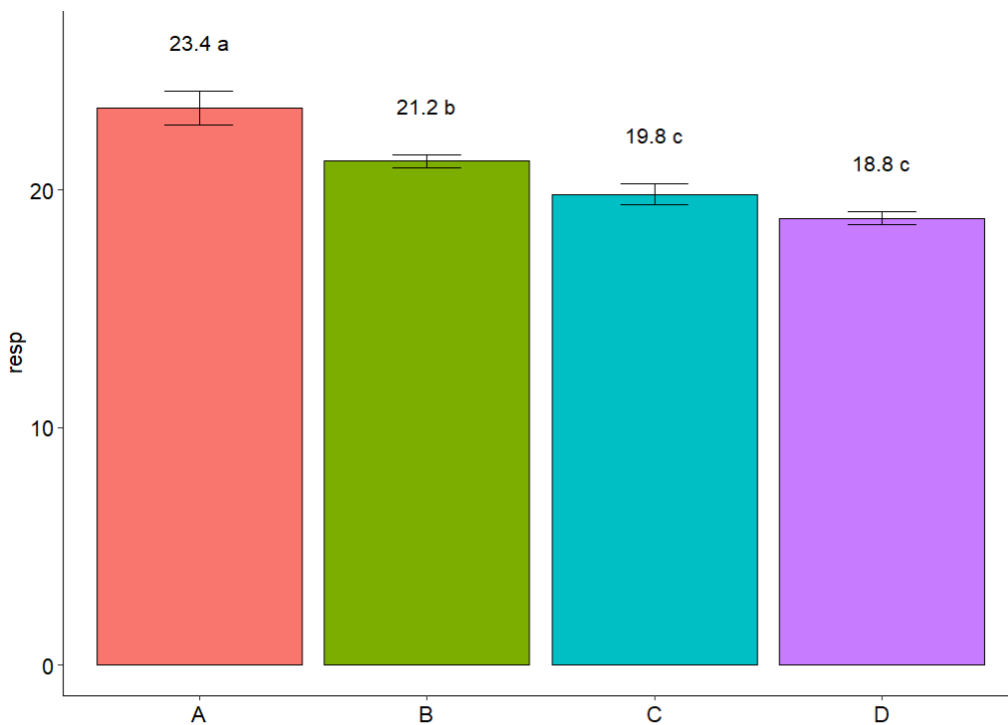
Residuals 8 1.65 0.206

- **p-value < 0.05** → **Reject H0** (means are not all equal)
- At least **two treatments are significantly different**

6. Tukey HSD Multiple Comparison

Treatment	Mean	Group
A	23.43	a
B	21.20	b
C	19.80	c
D	18.80	c

- Treatments with **different letters differ significantly**
- **A > B > C = D**
- Treatments C and D are **not significantly different** (both 'c')



p-value < 1e-04 ; CV = 2.18 %

R Language

```
datarbd <- read.csv("rbd new data.csv")
```

```
with(datarbd, DBC(trat, block, resp, addmean=TRUE, ylab="resp", fill="trat", quali=TRUE))
```

Interpretation are same as before.

DAY 3

```
setwd("G:\\Lesson\\Ongoing\\R Language\\Week 4\\Materials\\Day 3")
```

```
list.files()
```

```
data <- read.csv("polynomial_regression_agriculture.csv")
```

```
install.packages("AgroR")
```

```
library("AgroR")
```

```
model1 <- lm(yield ~ dose, data = data)
```

```
summary(model1)
```

```
model2 <- lm(yield ~ dose + I(dose^2), data = data)
```

```
summary(model2)
```

```
anova(model1, model2)
```

#model1 and model2 are significantly different as the p value is less than 0.05 and model 2 is significantly better.

```
model3 <- lm(yield ~ dose + I(dose^2) + I(dose^3), data = data)
```

```
summary(model3)
```

```
anova(model2, model3)
```

R Language